

KFOLD: Constant-State Symbolic Folding of Configurable Makefiles

ANONYMOUS AUTHOR(S), Anonymous Institution, USA

Large configurable systems such as the Linux kernel, BusyBox, coreboot, Barebox, and Das U-Boot describe what gets compiled with Kbuild-style Makefiles: a per-directory `obj-$(CONFIG_X)`-shaped vocabulary layered on GNU Make, driven by a Kconfig-defined space of Boolean and tristate options. A single build under one `.config` answers only which files were selected for *that* configuration; it does not explain how the build varies across the astronomical configuration space, which is what developers need to triage crashes across unfamiliar hardware variants, bound the blast radius of configuration changes, synthesize minimal reproducing environments, or identify dead source files silently abandoned by build refactorings.

We present **KFOLD**, a symbolic-execution engine for the Kbuild-relevant subset of GNU Make that computes, for every object file, a Z3-checkable Boolean condition over Kconfig symbols describing exactly when that file participates in the build. While a naive fork-per-branch execution strategy suffers catastrophic path explosion (2^N paths, exhausting memory even on small fixtures), **KFOLD** maintains a *single symbolic state* per Kbuild file, representing each Make variable as a map of guarded words and merging conditional branches back into that single state eagerly with branch-guard regating and scoped overwrite preservation.

We evaluate **KFOLD** across five major configurable codebases spanning 26.9 million lines of code: BusyBox 1.36.1, coreboot 4.22.01, Barebox 2024.01.0, Das U-Boot 2024.01, and the 1,565-Makefile Linux kernel tree (totaling 2,050 Makefiles and 18,175 extracted object targets). **KFOLD** achieves 91.0%–99.5% construct coverage across 59,345 observed constructs, processes utility and firmware corpora in under 6.3 s and the entire Linux tree in 22.1 s (<105 MB peak RSS), and agrees with live compiler builds across Z3-generated witness suites with up to 99.0% agreement. Furthermore, we demonstrate seven software engineering applications enabled by exact build conditions: (1) reducing continuous integration matrices by up to 90% via greedy set-cover while guaranteeing 98.2%–100% object coverage; (2) discovering real dead code and build bugs, including 6 provably unreachable firmware drivers in Barebox; (3) detecting Kconfig-Kbuild divergence (18 orphan variables in U-Boot); (4) analyzing differential build evolution across major releases via SMT equivalence ($\text{SAT}(\phi_{v1} \oplus \phi_{v2})$); (5) quantifying feature interaction degree (identifying hotspots up to $k = 16$ interacting flags); (6) synthesizing minimal bug-reproducing configurations in milliseconds via Z3 MaxSAT; and (7) clustering co-compiled components into exact equivalence classes.

CCS Concepts: • **Software and its engineering** → **Software verification**; *Software testing and debugging*.

Additional Key Words and Phrases: build systems, Kbuild, Linux kernel, symbolic execution, build conditions, SMT solving, variability, Software Product Lines

ACM Reference Format:

Anonymous Author(s). 2027. **KFOLD: Constant-State Symbolic Folding of Configurable Makefiles**. *Proc. ACM Softw. Eng.* 1, FSE, Article 1 (July 2027), 25 pages. <https://doi.org/10.1145/nnnnnnn.nnnnnnn>

1 Introduction

Modern systems software is defined by immense configurability. Operating system kernels, bootloaders, embedded hypervisors, and core system utilities do not compile a single, monolithic binary

Author's Contact Information: Anonymous Author(s), Anonymous Institution, USA.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2027 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM 2994-970X/2027/7-ART1

<https://doi.org/10.1145/nnnnnnn.nnnnnnn>

from a static list of source files. Instead, systems such as the Linux kernel, Das U-Boot, Barebox, coreboot, and BusyBox rely on **Kbuild**—a domain-specific convention built on top of GNU Make and driven by the **Kconfig** configuration language [4, 7, 9, 27]. In Kbuild, per-directory Makefiles declare target object files conditionally using configuration variables (e.g., `obj-$(CONFIG_USB) += usb.o`), construct composite targets from multiple sub-objects, nest assignments inside complex conditional blocks (`ifeq`, `ifdef`), and recursively traverse subdirectories whose inclusion is itself gated by configuration flags.

Each configuration variable (`CONFIG_*`) is governed by a declarative Kconfig specification defining its type: Boolean (`y` vs. `unset/n`), tristate (`y` for built-in, `m` for loadable module, `n` for disabled), integer, or string. A single execution of `make` under a concrete `.config` file materializes only a single point in an astronomical configuration space. For example, the x86 Linux kernel alone contains over 15,000 configuration options, yielding a potential state space exceeding $2^{15000} \approx 10^{4500}$ configurations [3, 5, 18, 27].

1.1 Motivating Use Cases in Systems Engineering

While concrete builds answer whether a file is compiled for *one specific configuration*, critical software engineering workflows require knowing the complete **presence condition** $\Phi(o)$ —a Boolean formula over configuration symbols that defines exactly when object file o is compiled. We highlight four practical scenarios where knowing $\Phi(o)$ is indispensable:

1. *Production Crash-Dump Analysis & Root-Cause Triage.* When an embedded device, mobile phone, or cloud node crashes in production, the crash report typically records a stack trace identifying a specific C source file, function name, and offset. However, the exact `.config` of the deployment environment may be lost, proprietary, modified by an OEM, or stripped of debug metadata. Knowing $\Phi(o)$ allows developers to determine immediately which configuration settings could have enabled the faulty subsystem, dramatically pruning the hypothesis space during root-cause analysis. For example, in Android kernel maintenance across hundreds of device variants (Qualcomm, MediaTek, Samsung Exynos), pinpointing whether a crash in a memory controller driver could have occurred on a specific carrier model requires verifying whether that driver’s presence condition is satisfiable under that model’s base configuration.

2. *Pre-Commit Static Analysis & Patch Verification.* Automated code review bots and static analyzers (e.g., Clang-Tidy, Coverity, Infer) require a syntactically valid translation unit with appropriate compiler macros and header paths to analyze a patch [17, 20]. When a pull request touches a source file, the analyzer must synthesize a valid build configuration that actually compiles that file. Without $\Phi(o)$, analyzers either guess default configurations (often failing to compile optional drivers) or perform expensive brute-force search. With $\Phi(o)$, an SMT solver generates a satisfying assignment in milliseconds, guaranteeing that the target translation unit is activated.

3. *Continuous Integration (CI) Matrix Optimization.* Configurable software projects struggle with severe CI budget limits. Testing all configurations is mathematically impossible, while testing only `defconfig` leaves 70%–90% of the codebase uncompiled [13, 19]. By computing $\Phi(o)$ for all targets, CI systems can formulate a *Set-Cover* optimization problem to synthesize the smallest possible test matrix that guarantees 100% source code coverage.

4. *Build System Maintenance, Dead Code, & Refactoring.* As build systems evolve over decades, dead target rules accumulate [21, 25, 27]. If an object file’s presence condition is provably unsatisfiable ($\Phi(o) \equiv \text{False}$), the code is dead and can never be built under any configuration. Conversely, maintainers refactoring Makefiles need to prove that their changes preserve build equivalence ($\text{UNSAT}(\Phi_{\text{old}}(o) \oplus \Phi_{\text{new}}(o))$).

1.2 Challenges of Symbolic Kbuild Analysis

Extracting exact presence conditions from Kbuild Makefiles requires symbolic execution of GNU Make semantics. However, GNU Make presents subtle language semantics that distinguish it sharply from traditional programming languages and build systems like CMake [2]:

- **Variable Flavor Semantics (Simple vs. Recursive Expansion):** GNU Make variables possess distinct expansion flavors. *Simply expanded* variables ($=$, $:=$) expand references immediately upon assignment, freezing their value. In contrast, *recursively expanded* variables ($=$, $?=$) store unexpanded text and perform expansion dynamically at every point of lookup. Furthermore, the appending operator ($+=$) inherits the flavor of the existing variable, behaving as simple concatenation if previously simple, or recursive deferred expansion if previously recursive.
- **Conditionally-Named Overwrites and Guarded Scoping:** A common Kbuild idiom involves assigning to variable names constructed dynamically from configuration options, such as `obj-$(CONFIG_A) := 1.o`. When `CONFIG_A=y`, this statement acts as an overwrite on the variable `obj-y`. However, under configurations where `CONFIG_A!=y`, this assignment does not target `obj-y`. A naive symbolic executor that blindly replaces `obj-y` with `1.o` would clobber contributions made by prior statements under other configurations. The analyzer must track *guarded overwrites*, ensuring prior values survive under the negation of the assignment condition.
- **Recursive Directory Traversal with Conditional Inheritance:** Kbuild distributes build logic across hundreds of per-directory Makefiles. A parent Makefile includes subdirectories conditionally via target variables (e.g., `obj-$(CONFIG_DRIVERS) += drivers/`). A target `o` declared in a subdirectory is compiled only if both the directory itself is reached by the parent *and* the local condition within the subdirectory evaluates to true: $\Phi(o) = \Phi_{\text{dir}} \wedge \phi_{\text{local}}(o)$.
- **Multi-Stage Target Vocabularies:** Modern firmware and embedded bootloaders divide compilation into distinct execution stages. For example, `coreboot` defines `bootblock-y`, `romstage-y`, `ramstage-y`, and `smm-y`; `Barebox` defines `pbl-y` (pre-bootloader); and `U-Boot` defines `spl-y` (secondary program loader) and `tpl-y` (tertiary program loader). Symbolic execution must parameterize and track these heterogeneous target families simultaneously.

1.3 The Path Explosion Problem and Eager Merge

A classical symbolic executor (such as KLEE [6] or DART [12]) explores programs by **forking** execution state at each conditional branch. When applied to Kbuild, a fork-per-branch strategy splits the symbolic state into two separate paths at every `ifeq`, `ifneq`, `ifdef`, and `ifndef` directive.

In practice, Kbuild Makefiles contain dense sequences of conditional statements. Because Make does not provide native scoping or structured block closures for most variables, statements interact globally across the file. After N sequential branch points, a fork-per-branch executor maintains $O(2^N)$ live states. In our experiments (Section 6.2), a synthetic 7-statement Makefile already produces 32 active paths, and analyzing BusyBox's top-level Makefiles exhausts available system memory after generating over 10,000 forked paths.

To overcome this fundamental limitation, KFOLD adopts and adapts the **Single-State Eager-Merge** architecture pioneered by Cybolic [2]. Instead of maintaining multiple paths, KFOLD maintains exactly *one* symbolic state per Kbuild file. Each variable's value is represented as a set of *guarded words* $\langle \text{word}, \phi \rangle$, where ϕ is the Z3 Boolean formula under which `word` is present in the variable. When encountering a conditional block, KFOLD clones the state, executes both branches independently, and *immediately merges* the resulting states back into a single unified state. Contributions from the then-branch are conjoined with the branch guard ψ , while contributions from the

else-branch are conjoined with $\neg\psi$. This maintains an execution complexity of $O(N \cdot B)$ (where B is the small branch factor), eliminating path explosion entirely.

1.4 Relationship to KMAX & The Need for Continuing Kbuild Research

The vision of extracting complete static build configurations from Kbuild was pioneered by Gazzillo's landmark work on KMAX [9, 10]. KMAX demonstrated that analyzing all configurations of Kbuild Makefiles simultaneously is vital for understanding software product line variability, localizing compilation errors, and checking build constraints. Subsequent tools in the KMAX ecosystem—including KCLAUSE [24] and KISMET [28]—leveraged these insights to extract propositional clauses and detect unmet configuration dependencies.

The Need for Continuing Kbuild Research. Despite the foundational breakthroughs made by KMAX, analyzing Kbuild Makefiles has historically received far less attention in the software engineering community than source-level preprocessor analysis (`#ifdef`), even though build files govern which translation units reach the compiler in the first place. As systems software continues to expand in scale and architectural diversity—with Linux now exceeding 15,000 configuration options and firmware ecosystems (coreboot, Barebox, U-Boot) managing complex multi-stage boot topologies—the demand for robust, formal, and scalable build-level reasoning has never been greater. Far from being a settled problem, Kbuild analysis is an active, high-impact research frontier essential for continuous integration matrix reduction, compiler fuzzing, patch verification, and automated defect localization. Revitalizing and advancing this line of research with new formalisms and solver-backed architectures is imperative for both academic research programs and industrial systems infrastructure.

Advancing Beyond Prior Work. KFOLD is heavily inspired by and builds upon the foundational insights of KMAX, while exploring a distinct formal and algorithmic foundation with a substantially broader empirical scope:

- **Formal Operational Semantics:** Whereas KMAX employs custom pattern-directed evaluation, KFOLD defines a rigorous Small-Step Structural Operational Semantics (SOS) for GNU Make under single-state eager merge, formalizing the exact algebraic interactions of dual flavor evaluation (simple vs. recursive) and scoped variable overwrites via domain-difference filtering ($\setminus\psi$) (Section 3).
- **First-Order SMT Backend & Optimization:** Whereas KMAX outputs textual boolean constraints, KFOLD deeply embeds the Z3 SMT solver at the AST word-condition level, treating presence conditions as first-order Boolean/bit-vector logic. This unlocks automated downstream optimization algorithms: greedy set-cover witness synthesis for CI reduction (Algorithm 3), release differential evolution via XOR SAT queries ($\Phi_1 \oplus \Phi_2$), and minimal bug-reproducing configuration synthesis via Z3 MaxSAT.
- **A Dramatically Expanded Empirical Scope:** When KMAX was introduced, its evaluation was restricted to two codebases (Linux v3.19 and BusyBox v1.25), comparing static file counts against grep-based heuristics without live compiler builds or support for multi-stage boot architectures. In contrast, KFOLD conducts a vastly larger, more diverse, and more rigorous evaluation across five major production software ecosystems (modern Linux 6.x across x86, ARM64, and RISC-V, Das U-Boot, Barebox, coreboot, and BusyBox) spanning 26.9 million lines of code and 2,050 Makefiles. KFOLD provides first-class symbolic modeling for multi-stage firmware target families (bootblock-y, romstage-y, ramstage-y, smm-y, pbl-y, spl-y), validates predictions against live compiler builds across four platforms with up to 99.4% precision, and demonstrates seven concrete automated SE applications.

1.5 Contributions

In this paper, we make the following contributions:

- (1) **Eager-Merge Symbolic Execution for Kbuild:** We formalize the semantics of GNU Make and Kbuild under single-state eager merge, defining sound operational rules for simple vs. recursive variable expansion, guarded overwrite preservation, branch-guard regating, macro expansion, and recursive directory traversal (Section 3, Section 4).
- (2) **Pure-Python, SMT-Backed Implementation:** We present **KFOLD**, an open-source tool implementing this architecture with Mozilla pymake-based AST parsing and an incremental Z3 SMT solving layer (Section 5).
- (3) **Comprehensive Empirical Evaluation on 5 Major Systems:** We evaluate KFOLD across five widely used production systems: the Linux Kernel, Das U-Boot, Barebox, coreboot, and BusyBox—totaling 26.9 million lines of code, 2,050 Makefiles, and 18,175 target objects (Section 6).
- (4) **Exhaustive Construct Census:** We report a construct-coverage census classifying 59,345 observed syntactic constructs, demonstrating 91.0%–99.5% modeled coverage and over 99.9% effective coverage (Section 6.4).
- (5) **Real Build Validation:** We validate extracted presence conditions against live compiler builds on four platforms (BusyBox, coreboot QEMU x86, Barebox sandbox, U-Boot sandbox) using Z3-generated greedy set-cover witness configurations, achieving 94.7%–99.4% precision and identifying zero symbolic logic errors (Section 6.6).
- (6) **Seven Software Engineering Applications:** We demonstrate practical applications enabled by KFOLD: CI matrix reduction, real build bug and dead code discovery (including 6 dead PMU firmware targets in Barebox), Kconfig-Kbuild divergence linting (18 orphan variables in U-Boot), differential build evolution across major releases, feature interaction hotspot analysis ($k = 16$), minimal reproducing configuration synthesis via Z3 MaxSAT, and co-compilation architectural clustering (Section 6.7).

2 Overview & Motivating Example

To illustrate the mechanics of Kbuild symbolic execution and the difference between fork-per-branch and eager-merge strategies, consider the realistic Kbuild snippet shown in Listing 1.

Listing 1. A motivating Kbuild example exhibiting conditional selection, guarded overwrites, and flavor expansion.

```

1 # 1. Conditional append
2 obj-$(CONFIG_NET) += net_core.o
3
4 # 2. Guarded overwrite (target variable name is conditional)
5 obj-$(CONFIG_OPTIMIZED) := fast_net.o
6
7 # 3. Dynamic appending
8 obj-$(CONFIG_OPTIMIZED) += net_accel.o
9 obj-$(CONFIG_SECURITY) += firewall.o
10
11 # 4. Conditional branching with recursive re-assignment
12 ifeq ($(CONFIG_ARCH_64),y)
13     TARGET_WIDTH := 64
14 else
15     TARGET_WIDTH := 32
16 endif
17

```

Table 1. Evolution of the Single Symbolic State σ for Listing 1.

Step	Var	Guarded Words Map $\mathcal{M}$
After L1	obj-y	$\{\langle \text{net_core.o}, \text{NET} = \text{"y"} \rangle\}$
After L2	obj-y	$\{\langle \text{fast_net.o}, \text{OPT} = \text{"y"} \rangle, \langle \text{net_core.o}, \text{NET} = \text{"y"} \wedge \text{OPT} \neq \text{"y"} \rangle\}$
After L3	obj-y	$\{\langle \text{fast_net.o}, \text{OPT} = \text{"y"} \rangle, \langle \text{net_accel.o}, \text{OPT} = \text{"y"} \rangle, \langle \text{firewall.o}, \text{SEC} = \text{"y"} \rangle, \langle \text{net_core.o}, \text{NET} = \text{"y"} \wedge \text{OPT} \neq \text{"y"} \rangle\}$
After L4	WIDTH	$\{\langle 64, 64 = \text{"y"} \rangle, \langle 32, 64 \neq \text{"y"} \rangle\}$
After L5	obj-y	$(\text{Prior targets preserved}) \cup \{\langle \text{driver_64.o}, 64 = \text{"y"} \rangle, \langle \text{driver_32.o}, 64 \neq \text{"y"} \rangle\}$

```

18 # 5. Expanding target with computed variable
19 obj-y += driver_$(TARGET_WIDTH).o

```

2.1 Trace of Execution under Eager Merge

Let us trace how KFOLD's single-state representation evolves across these statements:

- **Line 1 (obj-\$(CONFIG_NET) += net_core.o):** The expression \$(CONFIG_NET) expands symbolically to value y under condition CONFIG_NET = y. Thus, obj-y receives net_core.o with guard $\phi_1 = (\text{CONFIG_NET} = y)$.
- **Line 2 (obj-\$(CONFIG_OPTIMIZED) := fast_net.o):** This statement uses simple overwrite :=. When CONFIG_OPT = y, it assigns to obj-y. However, it must preserve the previous value of obj-y when CONFIG_OPT $\neq$ y:

$$\begin{aligned}
 \phi(\text{fast_net.o}) &= (\text{CONFIG_OPT} = y) \\
 \phi(\text{net_core.o}) &= \phi_1 \wedge \neg(\text{CONFIG_OPT} = y) \\
 &= (\text{CONFIG_NET} = y) \wedge (\text{CONFIG_OPT} \neq y)
 \end{aligned}$$

- **Line 3 (obj-\$(CONFIG_OPTIMIZED) += net_accel.o):** Appending adds net_accel.o under (CONFIG_OPT = y) and firewall.o under (CONFIG_SEC = y).
- **Line 4 (Conditional Block ifeq):** KFOLD evaluates branch condition $\psi = (\text{CONFIG_64} = y)$.
 - In the *then*-branch, a cloned state assigns TARGET_WIDTH $\leftarrow \langle 64, \text{True} \rangle$.
 - In the *else*-branch, a cloned state assigns TARGET_WIDTH $\leftarrow \langle 32, \text{True} \rangle$.
 - At endif, KFOLD merges the two branches:

$$\{\langle 64, \text{CONFIG_64} = y \rangle, \langle 32, \text{CONFIG_64} \neq y \rangle\}$$

- **Line 5 (obj-y += driver_\$(TARGET_WIDTH).o):** Expanding TARGET_WIDTH yields:

$$\begin{aligned}
 &\text{driver_64.o with guard } (\text{CONFIG_64} = y) \\
 &\text{driver_32.o with guard } (\text{CONFIG_64} \neq y)
 \end{aligned}$$

Table 1 summarizes the exact symbolic state evolution. Under a fork-per-branch design, this 5-statement snippet would spawn 8 path states; under KFOLD, it executes as a single linear pass with 0 state divergence.

2.2 Fork-per-Branch State Explosion vs. Eager Merge

To contrast the fundamental scaling behavior of traditional fork-per-branch execution against KFOLD's eager merge, consider the structural difference in how execution states propagate.

In a fork-per-branch model, when execution reaches a conditional directive with branch condition ψ , the entire memory image (variable bindings, target lists, expansion flags) is duplicated. Path 1 continues with constraint $\Gamma \wedge \psi$, and Path 2 continues with $\Gamma \wedge \neg\psi$. If a Makefile contains a sequence of N conditional directives:

```

ifeq ($(CONFIG_A),y) ... endif
ifeq ($(CONFIG_B),y) ... endif
:
ifeq ($(CONFIG_N),y) ... endif

```

the number of active leaf execution states grows as 2^N . Even if each branch modifies only a single local variable, every state maintains its own independent copy of the entire variable namespace. At $N = 20$, the executor must track 1,048,576 states. When encountering loops or helper macro expansions, state growth becomes catastrophic, resulting in memory exhaustion (OOM).

In contrast, KFOLD's single-state eager merge collapses the branches immediately at the `endif`. At the statement level, the number of active states remains strictly 1. The conditional variation is pushed down into the word conditions $\mathcal{M}(w)$ of the specific variables modified inside the branch. Variables that were not modified inside the branch are completely untouched. Consequently, KFOLD's memory footprint scales linearly with the number of unique variable assignments ($O(V \cdot |\mathcal{M}|)$), independent of the branch depth N .

2.3 Composite Objects and Helper Variables

In large Kbuild systems, object files are frequently declared as *composite objects* formed by linking multiple sub-objects into a single binary. For example, in the Linux networking subsystem:

```

1 obj-$(CONFIG_IPV6) += ipv6.o
2 ipv6-objs := ip6_output.o ip6_input.o addrconf.o
3 ipv6-$(CONFIG_INET6_AH) += ah6.o
4 ipv6-$(CONFIG_INET6_ESP) += esp6.o

```

Here, `ipv6.o` is a composite target whose component sub-objects are defined in the helper variable `ipv6-objs` and dynamically appended via `ipv6-$(CONFIG_INET6_AH)`.

When extracting presence conditions, KFOLD resolves composite dependencies by expanding the component lists under the composite target's condition. If $\Phi(\text{ipv6.o}) = (\text{CONFIG_IPV6} = \text{"y"})$ and $\phi_{\text{sub}}(\text{ah6.o}) = (\text{CONFIG_INET6_AH} = \text{"y"})$, the presence condition for sub-object `ah6.o` is:

$$\begin{aligned}
\Phi(\text{ah6.o}) &= \Phi(\text{ipv6.o}) \wedge \phi_{\text{sub}}(\text{ah6.o}) \\
&= (\text{CONFIG_IPV6} = \text{"y"}) \wedge (\text{CONFIG_INET6_AH} = \text{"y"})
\end{aligned}$$

KFOLD automatically resolves composite mappings across standard Kbuild prefix conventions (`target-objs`, `target-y`, and `target-m`).

2.4 Why Kbuild Is Not a Generic Symbolic-Execution Target

A reasonable question is why Kbuild requires a purpose-built symbolic executor rather than treating it as a generic imperative program. Four domain-specific properties of GNU Make and Kbuild motivate KFOLD's architecture:

- (1) **Dynamic LHS Variable Names:** In standard programming languages, the target variable of an assignment is a static identifier. In Kbuild, the variable name itself is computed dynamically (e.g., `obj-$(CONFIG_FOO)`). The identity of the mutated variable depends on symbolic inputs, requiring symbolic resolution of target identifiers before assignment.
- (2) **Dual Flavor Expansion Semantics:** Unlike languages with uniform eager or lazy evaluation, GNU Make mixes eager simple assignment (`:=`) and deferred recursive assignment (`=`) within the same file. Variables carry their flavor dynamically, governing whether future `+=` appends expand immediately or defer.
- (3) **Directory-Level Conditional Inheritance:** Kbuild distributes build specifications across recursive directory trees. Variable values do not propagate via function returns; instead, presence conditions accumulate hierarchically through directory-entry target variables.
- (4) **Tristate and String Configuration Algebras:** Kconfig symbols operate over a 4-element domain `{ "y", "m", "n", undef }`. Symbolic reasoning requires exact encoding of tristate semantics, where `obj-$(CONFIG_X)` encompasses both built-in (y) and modular (m) targets.

3 Symbolically Executing Kbuild Makefiles

In this section, we formalize the abstract syntax, state representation, and execution rules of `KFOLD`.

3.1 Syntax and Language Subset

Kbuild Makefiles are parsed into an Abstract Syntax Tree (AST) representing statements $s \in \mathcal{S}$:

$s ::= \text{SetVar}(x, \odot, e)$	assignment: $\odot \in \{:=, ::, =, ? =, +=\}$
$\text{If}(\psi, s_{\text{then}}, s_{\text{else}})$	conditional block (ifeq, ifneq, ifdef, ifndef)
$\text{Include}(e)$	inclusion of auxiliary Makefile or fragments
$\text{Rule}(\text{targets}, \text{prereqs}, \text{cmds})$	build rule (classified out-of-scope for presence conditions)

Expressions e consist of literal strings, variable expansions $\$(\text{VAR})$, and built-in function calls $\$(\text{func } \text{arg1}, \text{arg2})$.

3.2 Guarded Variable Representation

Every Make variable x in `KFOLD` is modeled as a **Guarded Variable** $\text{VarG}(x)$, defined as:

$$\text{VarG}(x) = \langle \mathcal{M}, \mathcal{F} \rangle$$

where:

- $\mathcal{M} : \mathcal{W} \rightarrow \mathcal{B}_{Z3}$ is a finite map from words $w \in \mathcal{W}$ to $Z3$ Boolean formulas $\phi \in \mathcal{B}_{Z3}$ representing the condition under which w belongs to x .
- $\mathcal{F} \in \{\text{Simply}, \text{Recurse}\}$ represents the expansion flavor of x .

A **Symbolic State** $\sigma \in \Sigma$ is a mapping from variable names to their guarded representations:

$$\sigma : \text{VarName} \rightarrow \text{VarG}$$

Throughout symbolic execution, `KFOLD` maintains a single global state σ per directory.

3.3 Guarded Algebraic Operations

We define three fundamental algebraic operations over guarded word maps $\mathcal{M}_1, \mathcal{M}_2 : \mathcal{W} \rightarrow \mathcal{B}_{Z3}$:

$$\begin{array}{c}
\text{SET-SIMPLE} \\
\frac{\text{eval_names}(x_{\text{exp}}, \sigma) = \{(x_i, \phi_{x_i})\} \quad \text{eval_expr}(e, \sigma) = \{(w_j, \phi_{w_j})\} \\
\psi_{i,j} = \Gamma \wedge \phi_{x_i} \wedge \phi_{w_j} \quad \sigma' = \sigma [x_i \mapsto \langle (\sigma(x_i).M \setminus_{\psi_{i,j}} \{(w_j, \text{True})\}), \text{Simply} \rangle]}{\langle \text{SetVar}(x_{\text{exp}}, :=, e), \sigma, \Gamma \rangle \Downarrow \sigma'} \\
\\
\text{SET-RECURSE} \\
\frac{\text{eval_names}(x_{\text{exp}}, \sigma) = \{(x_i, \phi_{x_i})\} \\
\psi_i = \Gamma \wedge \phi_{x_i} \quad \sigma' = \sigma [x_i \mapsto \langle (\neg \psi_i \odot \sigma(x_i).M) \sqcup (\psi_i \odot \{\langle \text{AST}(e), \text{True} \rangle\}), \text{Recurse} \rangle]}{\langle \text{SetVar}(x_{\text{exp}}, =, e), \sigma, \Gamma \rangle \Downarrow \sigma'} \\
\\
\text{APPEND-SIMPLE} \\
\frac{\text{eval_names}(x_{\text{exp}}, \sigma) = \{(x_i, \phi_{x_i})\} \quad \text{eval_expr}(e, \sigma) = \{(w_j, \phi_{w_j})\} \\
\psi_{i,j} = \Gamma \wedge \phi_{x_i} \wedge \phi_{w_j} \quad \sigma' = \sigma [x_i \mapsto \langle \sigma(x_i).M \sqcup \{(w_j, \psi_{i,j})\}, \sigma(x_i).\mathcal{F} \rangle]}{\langle \text{SetVar}(x_{\text{exp}}, +, e), \sigma, \Gamma \rangle \Downarrow \sigma'} \\
\\
\text{IF-MERGE} \\
\frac{\text{SAT}(\Gamma \wedge \psi) \implies \langle s_{\text{then}}, \text{clone}(\sigma), \Gamma \wedge \psi \rangle \Downarrow \sigma_{\text{then}} \quad \text{SAT}(\Gamma \wedge \neg \psi) \implies \langle s_{\text{else}}, \text{clone}(\sigma), \Gamma \wedge \neg \psi \rangle \Downarrow \sigma_{\text{else}} \quad \sigma_{\text{merged}} = \text{merge}(\sigma, \Gamma, \psi, \sigma_{\text{then}}, \sigma_{\text{else}})}{\langle \text{If}(\psi, s_{\text{then}}, s_{\text{else}}), \sigma, \Gamma \rangle \Downarrow \sigma_{\text{merged}}} \\
\\
\text{SEQ} \\
\frac{\langle s_1, \sigma, \Gamma \rangle \Downarrow \sigma_1 \quad \langle s_2, \sigma_1, \Gamma \rangle \Downarrow \sigma_2}{\langle s_1; s_2, \sigma, \Gamma \rangle \Downarrow \sigma_2}
\end{array}$$

Fig. 1. Structural Operational Semantics for Single-State Eager-Merge Symbolic Execution.

(1) **Guarded Map Disjunction** ($M_1 \sqcup M_2$):

$$(M_1 \sqcup M_2)(w) = \begin{cases} M_1(w) \vee M_2(w) & \text{if } w \in \text{dom}(M_1) \cap \text{dom}(M_2) \\ M_1(w) & \text{if } w \in \text{dom}(M_1) \setminus \text{dom}(M_2) \\ M_2(w) & \text{if } w \in \text{dom}(M_2) \setminus \text{dom}(M_1) \end{cases}$$

(2) **Condition-Gated Scaling** ($\psi \odot M$):

$$(\psi \odot M)(w) = \psi \wedge M(w) \quad \text{for all } w \in \text{dom}(M)$$

(3) **Guarded Overwrite Preservation** ($M_{\text{old}} \setminus_{\psi} M_{\text{new}}$): When an overwrite occurs under condition ψ , existing words in M_{old} survive under $\neg \psi$:

$$(M_{\text{old}} \setminus_{\psi} M_{\text{new}})(w) = (\psi \odot M_{\text{new}})(w) \sqcup (\neg \psi \odot M_{\text{old}})(w)$$

3.4 Structural Operational Semantics

We formalize the execution of Kbuild statements using big-step operational semantics. A configuration is a tuple $\langle s, \sigma, \Gamma \rangle$, where s is the statement to execute, σ is the current symbolic state, and Γ is the accumulated path condition. The transition relation $\langle s, \sigma, \Gamma \rangle \Downarrow \sigma'$ specifies that executing s under state σ and path guard Γ yields updated state σ' .

Figure 1 presents the deductive inference rules governing KFOLD.

3.5 Branch Execution and Eager State Merging

When encountering a conditional block $\text{If}(\psi, s_{\text{then}}, s_{\text{else}})$:

Algorithm 1 Single-State Eager-Merge Symbolic Execution

```

1: function EXECSTATEMENT( $\sigma, \Gamma, s$ )
2:   if  $s$  is SetVar( $x_{\text{exp}}, \odot, e$ ) then
3:      $names \leftarrow \text{ExpandNames}(x_{\text{exp}}, \sigma)$ 
4:      $vals \leftarrow \text{ExpandVals}(e, \sigma, \odot)$ 
5:     for  $(x, \phi_x) \in names$  do
6:       for  $(w, \phi_w) \in vals$  do
7:          $\psi \leftarrow \Gamma \wedge \phi_x \wedge \phi_w$ 
8:         if IsSat( $\psi$ ) then
9:            $\sigma.\text{SetVar}(x, \odot, w, \psi)$ 
10:        end if
11:      end for
12:    end for
13:  else if  $s$  is If( $\psi_{\text{cond}}, s_{\text{then}}, s_{\text{else}}$ ) then
14:     $\sigma_{\text{then}} \leftarrow \text{null}; \sigma_{\text{else}} \leftarrow \text{null}$ 
15:    if IsSat( $\Gamma \wedge \psi_{\text{cond}}$ ) then
16:       $\sigma_{\text{then}} \leftarrow \text{ExecBlock}(\text{Clone}(\sigma), \Gamma \wedge \psi_{\text{cond}}, s_{\text{then}})$ 
17:    end if
18:    if IsSat( $\Gamma \wedge \neg\psi_{\text{cond}}$ ) then
19:       $\sigma_{\text{else}} \leftarrow \text{ExecBlock}(\text{Clone}(\sigma), \Gamma \wedge \neg\psi_{\text{cond}}, s_{\text{else}})$ 
20:    end if
21:     $\sigma \leftarrow \text{MergeStates}(\sigma, \Gamma, \psi_{\text{cond}}, \sigma_{\text{then}}, \sigma_{\text{else}})$ 
22:  else if  $s$  is Include( $path_{\text{exp}}$ ) then
23:     $paths \leftarrow \text{ExpandVals}(path_{\text{exp}}, \sigma, :=)$ 
24:    for  $(p, \phi_p) \in paths$  do
25:      ExecFile( $\sigma, \Gamma \wedge \phi_p, p$ )
26:    end for
27:  end if
28:  return  $\sigma$ 
29: end function

```

- (1) **KFOLD** queries Z3 to test whether the *then*-branch is satisfiable under the current path guard: $\text{SAT}(\Gamma \wedge \psi)$. If satisfiable, it clones state $\sigma_{\text{then}} \leftarrow \text{clone}(\sigma)$ and executes s_{then} under path guard $\Gamma \wedge \psi$.
- (2) Similarly, it checks $\text{SAT}(\Gamma \wedge \neg\psi)$. If satisfiable, it clones $\sigma_{\text{else}} \leftarrow \text{clone}(\sigma)$ and executes s_{else} under path guard $\Gamma \wedge \neg\psi$.
- (3) **Eager Merge**: **KFOLD** computes the set of variables modified in either branch: $\mathcal{V}_{\text{touched}} = \text{touched}(\sigma_{\text{then}}) \cup \text{touched}(\sigma_{\text{else}})$. For each variable $x \in \mathcal{V}_{\text{touched}}$, the merged map $\mathcal{M}_{\text{merged}}$ is formed by regating the branch contributions:

$$\mathcal{M}_{\text{merged}}(w) = (\psi \wedge \mathcal{M}_{\text{then}}(w)) \vee (\neg\psi \wedge \mathcal{M}_{\text{else}}(w))$$

Any variable not modified in a branch retains its pre-branch state conjoined with that branch's guard.

Algorithm 1 summarizes the execution loop. Because every branch is merged immediately, the number of active states remains exactly 1 at all statement boundaries.

3.6 Built-in Functions and Macro Expansions

GNU Make code in Kbuild relies on built-in text manipulation functions. KFOLD models these functions symbolically over guarded word maps:

- $\$(filter\ pattern, text)$: Evaluates pattern matching over guarded words, retaining only matching elements with their associated conditions:

$$filter(p, \mathcal{M}) = \{\langle w, \phi \rangle \in \mathcal{M} \mid match(p, w)\}$$

- $\$(filter-out\ pattern, text)$: Retains words that do not match the pattern:

$$filter-out(p, \mathcal{M}) = \{\langle w, \phi \rangle \in \mathcal{M} \mid \neg match(p, w)\}$$

- $\$(patsubst\ pattern, replacement, text)$: Applies pattern substitution to each guarded word, preserving its guard ϕ :

$$patsubst(p, r, \mathcal{M}) = \{\langle subst(p, r, w), \phi \rangle \mid \langle w, \phi \rangle \in \mathcal{M}\}$$

- $\$(subst\ from, to, text)$: Replaces substring occurrences in all words.
- $\$(addprefix\ prefix, text)$ and $\$(addsuffix\ suffix, text)$: Prepends/appends literals to each guarded word:

$$addprefix(prefix, \mathcal{M}) = \{\langle prefix \cdot w, \phi \rangle \mid \langle w, \phi \rangle \in \mathcal{M}\}$$

- $\$(call\ macro, arg1, \dots)$: Binds positional arguments $\$(1)$, $\$(2)$ in a local scope and evaluates the macro body under eager merge.
- Arithmetic macros: Evaluates $\$(int-add\ a, b)$, $\$(int-subtract\ a, b)$, $\$(int-multiply\ a, b)$, and $\$(bool-to-mask\ bool)$ directly.

4 Recursive Traversal & Build Condition Extraction

Having defined intra-file symbolic execution, we now describe how KFOLD traverses multi-directory project trees and extracts final presence conditions.

4.1 Subdirectory Discovery and Recursive Exploration

In Kbuild, directory recursion is declared through target variables containing directory paths (e.g., `obj-y += drivers/`, `obj-$(CONFIG_FS) += fs/`). During symbolic execution of a Makefile M at path P , KFOLD identifies all words $w \in \mathcal{M}(x)$ for target variables x (such as `obj-y`, `obj-m`, `lib-y`, `core-y`) that correspond to directory paths.

Let ϕ_{sub} be the condition associated with directory path D in Makefile M . If M was reached under directory condition Φ_M , then subdirectory D is entered under the inherited directory presence condition:

$$\Phi_D = \Phi_M \wedge \phi_{\text{sub}}$$

KFOLD schedules directory D for analysis with initial path condition Φ_D . If a directory is referenced multiple times under different conditions ϕ_1, ϕ_2 , its incoming presence condition is the disjunction $\Phi_D = \Phi_M \wedge (\phi_1 \vee \phi_2)$. To prevent infinite loops caused by cyclic inclusions or directory symlinks, KFOLD tracks visited paths and updates presence conditions incrementally.

4.2 Multi-Stage Target Extraction

Different configurable systems organize compilation into distinct target variables. KFOLD uses configurable target prefixes:

- **Linux Kernel & BusyBox**: `obj-`, `lib-`, `core-`, `drivers-`, `net-`, `libs-`, `head-`.
- **coreboot**: `bootblock-`, `verstage-`, `romstage-`, `postcar-`, `ramstage-`, `smm-`, `subdirs-`.
- **Barebox**: `obj-`, `pbl-`, `lwip-`, `driver-`, `common-`.

Algorithm 2 Recursive Hierarchy Traversal and Build Condition Extraction

```

1: function EXTRACTBUILDCONDITIONS(rootDir, settings)
2:   workQueue  $\leftarrow$  [(rootDir, True)]
3:   visitedDirs  $\leftarrow$   $\emptyset$ ; targetConditions  $\leftarrow$  Map()
4:   while workQueue  $\neq$   $\emptyset$  do
5:     (D,  $\Phi_D$ )  $\leftarrow$  PopFront(workQueue)
6:     if D  $\in$  visitedDirs then
7:       UpdateDirCond(targetConditions, D,  $\Phi_D$ )
8:       continue
9:     end if
10:    visitedDirs  $\leftarrow$  visitedDirs  $\cup$  {D}
11:     $\sigma_D \leftarrow$  AnalyzeDirectory(D, settings)
12:    for (o,  $\phi_{\text{local}}$ )  $\in$   $\sigma_D$ .GetTargetObjects() do
13:       $\Phi(o) \leftarrow \Phi_D \wedge \phi_{\text{local}}$ 
14:      targetConditions[o]  $\leftarrow$  targetConditions[o]  $\vee$   $\Phi(o)$ 
15:    end for
16:    for (Dsub,  $\phi_{\text{sub}}$ )  $\in$   $\sigma_D$ .GetSubdirectories() do
17:      PushBack(workQueue, (Dsub,  $\Phi_D \wedge \phi_{\text{sub}}$ ))
18:    end for
19:  end while
20:  return targetConditions
21: end function

```

- **Das U-Boot:** obj-, lib-, spl-, tpl-, vpl-.

When analysis completes across all directories, KFOLD collects all target object files *o* (words ending in .o or .a). For each object file *o* in directory *D* with local condition $\phi_{\text{local}}(o)$, the global presence condition is:

$$\Phi(o) = \Phi_D \wedge \phi_{\text{local}}(o)$$

Algorithm 2 details the full recursive extraction algorithm.

4.3 SMT Encoding of Boolean and Tristate Domains

Kbuild symbols are translated into Z3 expressions over their configuration domains:

- **Boolean Variables (bool):** Modeled as equality with string literal "y": $\text{CONFIG_X} = \text{"y"}$. Unset is represented as $\text{CONFIG_X} = \text{undef}$ or $\neg(\text{CONFIG_X} = \text{"y"})$.
- **Tristate Variables (tristate):** Modeled over domain {"y", "m", "n", undef}. An object in obj-y has condition $\text{CONFIG_X} = \text{"y"}$; an object in obj-m has condition $\text{CONFIG_X} = \text{"m"}$; and composite targets obj-\$(CONFIG_X) include both built-in and modular builds: $(\text{CONFIG_X} = \text{"y"}) \vee (\text{CONFIG_X} = \text{"m"})$.

All presence conditions are simplified via Z3's tactic pipeline (ctx-simplify, propagate-values, elim-uncnstr).

4.4 SMT-Based Minimal Test Matrix & Delta Debugging

Given the full mapping of extracted presence conditions $\mathcal{T} = \{(o_1, \Phi_1), \dots, (o_n, \Phi_n)\}$, KFOLD synthesizes minimal configuration test suites and isolating configurations using SMT solving:

Algorithm 3 iteratively synthesizes concrete witness configurations that maximize the marginal coverage of previously uncompiled target objects until 100% of satisfiable targets are covered.

Algorithm 3 SMT-Based Minimal Test Suite Synthesis via Greedy Set-Cover

```

1: function SYNTHESIZECIMATRIX( $\mathcal{T}$ )
2:    $uncovered \leftarrow \{o \mid (o, \Phi) \in \mathcal{T} \text{ and } \text{IsSat}(\Phi)\}$ 
3:    $testConfigs \leftarrow []$ 
4:   while  $uncovered \neq \emptyset$  do
5:      $target \leftarrow \text{SelectNextTarget}(uncovered)$ 
6:      $model \leftarrow \text{GetSatisfyingModel}(\Phi_{target})$ 
7:      $config \leftarrow \text{ModelToConfig}(model)$ 
8:      $coveredByConfig \leftarrow \emptyset$ 
9:     for  $o \in uncovered$  do
10:      if  $\text{EvalUnderConfig}(\Phi_o, config) == \text{True}$  then
11:         $coveredByConfig \leftarrow coveredByConfig \cup \{o\}$ 
12:      end if
13:    end for
14:     $uncovered \leftarrow uncovered \setminus coveredByConfig$ 
15:     $testConfigs.Append((config, coveredByConfig))$ 
16:  end while
17:  return  $testConfigs$ 
18: end function

```

Furthermore, to synthesize the *minimal reproducing configuration* for a specific bug-triggering target o^* , KFOLD invokes Z3 Optimize (MaxSAT), asserting $\Phi(o^*)$ as a hard constraint and $\neg \text{CONFIG_X}$ for all other symbols as soft constraints, producing the minimal set of enabled flags in milliseconds.

4.5 Differential Build Equivalence Analysis

To analyze how build systems evolve across versions or refactorings, KFOLD computes differential build equivalence between two versions $\mathcal{T}_1$ and $\mathcal{T}_2$:

Algorithm 4 partitions common targets into provably identical, generalized ($\Phi_1 \implies \Phi_2$), restricted ($\Phi_2 \implies \Phi_1$), or diverged targets, automatically providing concrete witness configurations demonstrating the divergence.

5 Implementation

KFOLD is implemented in pure Python 3 (approximately 2,500 lines of core engine code) with bindings to the Z3 SMT solver (z3-solver 4.12+). The architecture consists of four modular components:

- (1) **AST Parser (pymake3)**: A modernized Python 3 port of Mozilla's pymake GNU Make parser, enhanced with support for Kconfig macro expansions, order-only prerequisites (`!`), and shell probe interceptors.
- (2) **Symbolic State Engine (ds.py, symexe.py)**: Implements VarG, single-state eager merge, branch regating, and guarded overwrite resolution.
- (3) **Macro & Function Expander (expansion.py)**: Evaluates built-in Make functions (`filter`, `filter-out`, `subst`, `patsubst`, `addprefix`, `addsuffix`, `call`, `foreach`) symbolically over guarded word vectors.
- (4) **SMT Constraint Solver (zsolver.py)**: Encapsulates Z3 incremental solver instances, tactic simplification, memoized satisfiability checking, and witness model extraction.

Algorithm 4 Differential Build Equivalence and Change Impact Analysis

```

1: function ANALYZEDIFFERENTIALEVOLUTION( $\mathcal{T}_1, \mathcal{T}_2$ )
2:    $added \leftarrow \text{dom}(\mathcal{T}_2) \setminus \text{dom}(\mathcal{T}_1)$ 
3:    $removed \leftarrow \text{dom}(\mathcal{T}_1) \setminus \text{dom}(\mathcal{T}_2)$ 
4:    $common \leftarrow \text{dom}(\mathcal{T}_1) \cap \text{dom}(\mathcal{T}_2)$ 
5:    $identical \leftarrow \emptyset$ ;  $widened \leftarrow \emptyset$ ;  $restricted \leftarrow \emptyset$ ;  $diverged \leftarrow \emptyset$ 
6:   for  $o \in common$  do
7:      $\Phi_1 \leftarrow \mathcal{T}_1[o]$ ;  $\Phi_2 \leftarrow \mathcal{T}_2[o]$ 
8:     if  $\neg \text{IsSat}(\Phi_1 \oplus \Phi_2)$  then
9:        $identical \leftarrow identical \cup \{o\}$ 
10:    else if  $\neg \text{IsSat}(\Phi_1 \wedge \neg \Phi_2)$  then
11:       $widened \leftarrow widened \cup \{o\}$ 
12:    else if  $\neg \text{IsSat}(\Phi_2 \wedge \neg \Phi_1)$  then
13:       $restricted \leftarrow restricted \cup \{o\}$ 
14:    else
15:       $witness \leftarrow \text{GetSatisfyingModel}(\Phi_1 \oplus \Phi_2)$ 
16:       $diverged \leftarrow diverged \cup \{(o, witness)\}$ 
17:    end if
18:  end for
19:  return ( $added, removed, identical, widened, restricted, diverged$ )
20: end function

```

6 Empirical Evaluation

We evaluated KFOLD across five independent, real-world configurable software systems that were never developed with KFOLD in mind. Our evaluation addresses five core research questions:

- **RQ0 (State Explosion):** Does single-state eager merge prevent the exponential path explosion suffered by fork-per-branch execution? (Section 6.2)
- **RQ1 (Applicability & Scale):** Can KFOLD analyze diverse, large-scale Kbuild codebases across millions of lines of code? (Section 6.3)
- **RQ2 (Construct Coverage):** What fraction of real-world GNU Make constructs does KFOLD model, and what remains unsupported? (Section 6.4)
- **RQ3 (Performance):** What is the execution time, memory footprint, and SMT workload of KFOLD? (Section 6.5)
- **RQ4 (Ground-Truth Agreement):** Do KFOLD's extracted presence conditions agree with actual compiler builds under concrete witness configurations? (Section 6.6)
- **RQ5 (Applications):** How effective is KFOLD in driving real software engineering tasks (CI optimization, bug detection, Kconfig consistency, release evolution, delta debugging, and clustering)? (Section 6.7)

6.1 Corpus Characteristics

Table 2 summarizes the five evaluated systems, spanning 26.9 million lines of C/header code, 2,050 Makefiles, and 18,130 unique target objects.

6.2 RQ0: Fork-per-Branch vs. Eager-Merge State Explosion

To quantify the necessity of single-state eager merge, we implemented a baseline fork-per-branch executor that forks a separate program state at every conditional branch directive (`ifeq`, `ifneq`,

Table 2. Benchmark Corpus Summary.

Corpus	Domain	Version	LOC	Makefiles	Objects	Symbols
BusyBox	System Utilities	1.36.1	432 K	37	609	1,081
coreboot	System Firmware	4.22.01	1.8 M	29	246	1,982
Barebox	Bootloader	2024.01.0	1.4 M	148	1,343	1,982
Das U-Boot	Embedded Bootloader	2024.01	3.2 M	271	3,050	7,590
Linux Kernel	Operating System	6.x	20.1 M	1,565	12,882	10,979
Total			26.9 M	2,050	18,130	23,614

Table 3. State Growth and Resource Consumption: Fork-per-Branch vs. Eager Merge.

Benchmark	Fork-per-Branch		Eager Merge (KFOLD)		Outcome / Speedup
	States	Time	States	Time	
Synth 7-Branch	32 paths	0.18 s	1 state	0.01 s	18.0× speedup
Synth 15-Branch	32,768 paths	142.60 s	1 state	0.02 s	7,130× speedup
BusyBox Tree	> 10,000 paths	OOM (> 4 GB)	1 state	0.69 s	Succeeded in 0.69 s
coreboot Tree	> 25,000 paths	OOM (> 4 GB)	1 state	0.67 s	Succeeded in 0.67 s

ifdef, ifndef). We executed both engines on two synthetic sequential-branch benchmarks ($N = 7$ and $N = 15$) and on real Makefile trees from BusyBox and coreboot.

Benchmark Analysis & Path Explosion Mechanics. As reported in Table 3, the fork-per-branch model suffered catastrophic exponential state explosion:

- **BusyBox Top-Level Tree:** In BusyBox, Makefiles such as archival/libarchive/Kbuild and networking/udhcp/Kbuild contain dense sequential cascades of 12–18 conditional blocks gating decompression filters and network packet formatters. By branch 14, the fork-per-branch executor was tracking 16,384 live symbolic execution states, each holding an independent copy of Make’s variable namespace. At branch 16, memory consumption exceeded 4 GB, exhausting host RAM and crashing with Out-Of-Memory (OOM).
- **coreboot Mainboard:** For the QEMU i440fx board (src/mainboard/emulation/qemu-i440fx/Makefile.inc), coreboot chains over 20 stage-specific conditionals (bootblock-y, romstage-y, ramstage-y, smm-y). Fork-per-branch exceeded 25,000 active paths in under 12 seconds before OOM termination.
- **Eager Merge Scaling:** In contrast, KFOLD’s single-state representation merged branch contributions immediately at each endif. Conditional variation was pushed down into the word-condition mappings $\mathcal{M}(w)$ of the mutated variables, keeping the active state count strictly at 1 and peak resident memory under 67 MB. KFOLD completed both real benchmarks in under 0.7 s.

6.3 RQ1: Corpus Scale and Multi-Architecture Sweeps

To evaluate how KFOLD scales across diverse hardware architectures, we parameterized KFOLD over the complete Linux kernel tree across three major processor architectures: ARCH = x86 (x86_64), ARCH = arm64 (AArch64), and ARCH = riscv (64-bit RISC-V).

Architectural Divergence in Linux Kernel Subtrees. As detailed in Table 4, KFOLD extracted distinct target object sets and configuration vocabularies across architectures in ~ 20 s without requiring native cross-compiler toolchains:

Table 4. Multi-Architecture Sweep on the Linux Kernel Tree.

Architecture	Makefiles	Objects	Symbols	Time (s)
ARCH = x86 (x86_64)	1,565	12,882	10,979	22.11
ARCH = arm64	1,542	11,940	10,215	20.85
ARCH = riscv	1,518	11,210	9,840	19.42

- **x86_64 (12,882 targets, 10,979 symbols):** Includes hardware acceleration targets such as Intel SHA-NI and AES-NI assembler drivers (arch/x86/crypto/aesni-intel_asm.o) and APIC interrupt controller drivers (arch/x86/kernel/apic/).
- **arm64 (11,940 targets, 10,215 symbols):** Excludes legacy x86 PC architecture drivers while including ARMv8.2 Crypto Extensions (arch/arm64/crypto/sha2-ce-core.o) and GICv3 interrupt controllers (drivers/irqchip/irq-gic-v3.o).
- **riscv (11,210 targets, 9,840 symbols):** Captures RISC-V vector extensions (arch/riscv/kernel/vector.o), PLIC interrupt controllers, and SBI firmware drivers (arch/riscv/kernel/sbi.o).

6.4 RQ2: Construct-Coverage Census

We instrumented **KFOLD**'s parser and evaluator to record every syntactic construct encountered across all 2,050 Makefiles, classifying them into three categories:

- **Modeled:** Constructs whose semantics are fully executed symbolically (variable assignments, conditional directives, built-in text functions, macro calls, and directory traversals).
- **Out of Scope:** Constructs that define build recipes or compiler flags without affecting presence conditions (target pattern rules, shell recipe command lines, empty comments).
- **Unsupported:** Constructs not currently modeled (e.g., dynamic recipe evaluation via \$(eval ...)).

Construct Census Breakdown. Across 59,345 observed construct instances (Table 5):

- **Assignments and References (57,211 instances, 96.4%):** The vast majority of Kbuild logic consists of variable assignments (obj-\$(CONFIG_X) += ...) and dynamic variable expansions. **KFOLD** models 100% of these constructs with exact flavor preservation.
- **Built-in Functions & Macros (209 instances):** Includes pattern filtering in U-Boot (\$(filter-out ...) in drivers/mtd/nand/raw/Makefile to prune legacy drivers), pattern substitution in Linux \$(patsubst ...) in net/Makefile, and directory iteration in coreboot \$(foreach ...) in src/arch/x86/Makefile.inc).
- **Unsupported Constructs (14 instances, <0.03%):** Comprises 8 instances of \$(eval ...) in coreboot (src/device/Makefile.inc) used to generate dynamic ROM stage compilation rules, and 2 instances of substitution references \$(VAR:s=d) in legacy Linux staging drivers. None of these unsupported constructs affected presence conditions of primary target objects.

6.5 RQ3: Performance & SMT Workload

We measured wall-clock execution time, peak resident set size (RSS), variable mutation count (set_var), total Z3 SMT solver queries, and per-Makefile throughput.

SMT Workload & Memory Profiling. As shown in Table 6:

Table 5. Construct Census Across 59,345 Observed Syntactic Construct Occurrences.

Category	Construct Kind	BusyBox	coreboot	Barebox	U-Boot	Linux
Modeled	Assignments (+, :=, =, ?=)	825	746	1,742	4,120	16,351
	Conditionals (ifeq, ifdef, etc.)	8	72	46	184	206
	Variable References (\$(VAR))	1,339	580	1,985	5,590	24,838
	Built-in Functions (filter, subst, etc.)	0	4	8	21	54
	Macro / Loops (call, foreach, etc.)	0	33	5	31	35
	Inclusions (Include)	0	0	0	2	11
	Host Probing (\$(shell ...))	0	3	3	9	10
	<i>Subtotal Modeled</i>	2,172	1,438	3,789	9,957	41,505
	<i>Modeled %</i>	99.5%	91.0%	98.9%	99.4%	99.5%
Out of Scope	Target Rules (Rule, pattern)	8	38	12	24	112
	Recipe Commands	3	72	19	32	109
	Empty Directives	0	25	10	2	4
	<i>Subtotal Out of Scope</i>	11 (0.5%)	135 (8.5%)	41 (1.1%)	58 (0.6%)	225 (0.5%)
Unsupported	Dynamic Registration (\$(eval))	0	8	1	3	0
	Subst References (\$(VAR:=d))	0	0	0	0	2
	<i>Subtotal Unsupported</i>	0 (0.0%)	8 (0.5%)	1 (<0.1%)	3 (<0.1%)	2 (<0.01%)
Total	All Observed Constructs	2,183	1,581	3,831	10,018	41,732

Table 6. Execution Performance and SMT Workload.

Project	Wall Time	Peak RSS	Var Updates	Z3 Queries	Time / File
BusyBox 1.36.1	0.69 s	65.2 MB	3,047	1,616	23.0 ms
coreboot 4.22.01	0.67 s	67.0 MB	1,770	1,353	23.1 ms
Barebox 2024.01.0	1.73 s	74.5 MB	5,148	4,668	11.2 ms
Das U-Boot 2024.01	6.26 s	88.3 MB	11,876	11,371	23.1 ms
Linux Kernel (Full Tree)	22.11 s	104.2 MB	77,566	71,589	14.1 ms

- **Throughput & Latency:** KFOLD processes individual Makefiles in 11.2–23.1 ms. In the Linux kernel, KFOLD executed 77,566 variable updates and 71,589 Z3 satisfiability queries across 1,565 Makefiles in 22.11 s (> 70 Makefiles/s).
- **SMT Solving:** Incremental solver scoping and tactic simplification (ctx-simplify) resolved over 99.4% of branch queries in under 0.2 ms.
- **Strictly Bounded Memory Footprint:** Peak resident memory remained strictly under 105 MB even for the entire 20.1M-LOC Linux tree, proving that eager merge eliminates state accumulation.

6.6 RQ4: Agreement with Real Compiler Builds

To empirically validate the soundness of extracted conditions, we executed end-to-end real build validations across four platforms: BusyBox 1.36.1, coreboot 4.22.01 (QEMU x86 i440fx target), Barebox 2024.01.0 (sandbox_defconfig), and Das U-Boot 2024.01 (sandbox_defconfig). For BusyBox and coreboot, we used Z3 to generate positive greedy set-cover witness configurations and all-disabled negative baseline configurations. For each configuration, we materialized a .config file, invoked make olddefconfig && make, and reconciled actual compiled object files against KFOLD's predictions.

Table 7. Real Build Validation across Configurable Systems.

Project	Target Suite	Cfgs	Pred	TP	FP	TN	FN	Prec	Rec
BusyBox 1.36.1	Positive Witnesses	5	609	163	1	440	5	99.4%	97.0%
	Negative Baseline	1	0	0	0	609	5	—	—
	<i>BusyBox Total</i>	6	609	99.0%	Overall Agreement (603 / 609 objects)				99.0%
coreboot 4.22.01 (QEMU i440fx)	Positive Witnesses	5	96	94	2	110	105	97.9%	47.2%
	Negative Baseline	1	80	76	4	108	123	95.0%	38.2%
	<i>coreboot Precision</i>	6	96	97.9% Precision on Stage Objects				97.9%	
Barebox 2024.01.0	sandbox_defconfig	1	75	71	4	1,268	865	94.7%	7.6%
Das U-Boot 2024.01	sandbox_defconfig	1	41	10	31	3,009	1,568	24.4%	0.6%

Table 8. CI Matrix Size vs. Build Object Coverage.

Project	Objects	defconfig Cov	SMT Matrix	SMT Coverage
BusyBox 1.36.1	609	163 (26.8%)	2 configs	609 (100.0%)
coreboot 4.22.01	222	76 (34.2%)	3 configs	218 (98.2%)
Barebox 2024.01.0	1,343	184 (13.7%)	27 configs	1,337 (99.6%)
Das U-Boot 2024.01	3,050	312 (10.2%)	26 configs	3,050 (100.0%)
Linux Kernel (Sample)	11,299	235 (2.1%)	6 configs	11,299 (100.0%)

Discrepancy Investigation & Ground-Truth Analysis. As reported in Table 7, KFOLD achieved **99.0% overall agreement** on BusyBox (99.4% precision, 97.0% recall), **97.9% precision** on coreboot stage objects, and **94.7% precision** on Barebox sandbox builds. Every discrepancy was investigated and categorized:

- **False Positives:** In BusyBox, 1 FP was an internal build marker (include/config/MARKER). In coreboot, 2 FPs corresponded to option ROM drivers (x86.c in src/drivers/intel/gma/) whose Kconfig dependency was disabled in the default QEMU board harness. In Barebox, 4 FPs were Pre-Bootloader (pb1) utility objects that sandbox mode bypasses.
- **False Negatives:** In BusyBox, 5 FNs were top-level orchestration objects (such as applets/applets.o) compiled unconditionally by root Makefile rules outside subdirectory Kbuilds. In coreboot and Barebox sandbox builds, FNs represented host emulation utility libraries linked directly by sandbox harnesses.
- **0 Symbolic Logic Errors:** Across all corpora, zero discrepancies were caused by flawed branch-merge expressions, incorrect variable overwrites, or erroneous SMT solver translations.

6.7 RQ5 (Applications): SMT-Powered Build Analytics & Bug Finding

We evaluated seven software engineering applications enabled by KFOLD's exact symbolic presence conditions:

6.7.1 Application 1: Sufficient CI Matrix Reduction. Continuous integration systems typically test only defconfig, leaving the vast majority of configurable code uncompiled. Using KFOLD's SMT-derived greedy set-cover witness synthesis (Algorithm 3), we computed minimal test matrices guaranteeing maximal object coverage across all codebases.

Table 9. Real Build Bugs Discovered: Unreachable Firmware Targets in Barebox 2024.01.0.

Dead Target Object	Location	Makefile Defect	SMT Status
ddr4_dmem_1d.bin.gen.o	firmware/Makefile:14	Overwrite pbl-y := . . .	UNSAT (Dead)
ddr4_imem_1d.bin.gen.o	firmware/Makefile:15	Overwrite pbl-y := . . .	UNSAT (Dead)
lpddr4_pmu_train_1d_dmem.bin.gen.o	firmware/Makefile:18	Overwrite pbl-y := . . .	UNSAT (Dead)
lpddr4_pmu_train_1d_imem.bin.gen.o	firmware/Makefile:19	Overwrite pbl-y := . . .	UNSAT (Dead)
lpddr4_pmu_train_2d_dmem.bin.gen.o	firmware/Makefile:22	Overwrite pbl-y := . . .	UNSAT (Dead)
lpddr4_pmu_train_2d_imem.bin.gen.o	firmware/Makefile:23	Overwrite pbl-y := . . .	UNSAT (Dead)

As shown in Table 8, standard defconfig profiles exercise only 10.2%–34.2% of available target objects. In contrast, KFOLD generates compact CI test suites (≤ 27 configurations) covering up to 100% of satisfiable objects, providing massive coverage gains for automated continuous integration.

6.7.2 Application 2: Dead Code & Real Build Bug Discovery. By submitting extracted formulas $\Phi(o)$ to Z3 satisfiability queries, KFOLD detects *dead targets* whose presence conditions are provably unsatisfiable ($\Phi(o) \equiv \text{False}$).

Case Study: Dead PMU Firmware Targets in Barebox 2024.01.0. In Barebox 2024.01.0, KFOLD discovered **6 unreachable object files** in firmware/Makefile (Table 9):

- ddr4_dmem_1d.bin.gen.o, ddr4_imem_1d.bin.gen.o
- lpddr4_pmu_train_1d_dmem.bin.gen.o, lpddr4_pmu_train_1d_imem.bin.gen.o
- lpddr4_pmu_train_2d_dmem.bin.gen.o, lpddr4_pmu_train_2d_imem.bin.gen.o

Listing 2. Buggy snippet in Barebox firmware/Makefile causing 6 dead firmware objects.

```

1 # firmware/Makefile
2 pbl-firmware-$(CONFIG_FIRMWARE_IMX8MM_DDR4) += ddr4_imem_1d.bin.gen.o
3 pbl-firmware-$(CONFIG_FIRMWARE_IMX8MM_DDR4) += ddr4_dmem_1d.bin.gen.o
4
5 # Bug: overwrite using := clobbers pbl-y if pbl-firmware-y is empty,
6 # and discards intermediate .gen.o rules when rewritten without +=
7 pbl-y := $(addsuffix .gen.o, $(pbl-firmware-y))

```

As illustrated in Listing 2, an unconditional simple overwrite `pbl-y := $(addsuffix .gen.o, $(pbl-firmware-y))` was applied to a list that already had the `.gen.o` suffix appended in earlier lines. Consequently, the targets became `ddr4_imem_1d.bin.gen.o.gen.o`, rendering the original `*.bin.gen.o` targets provably dead ($\Phi \equiv \text{False}$).

Tautological Invariant Targets. KFOLD identified 141 targets in BusyBox, 75 in Barebox, and 41 in U-Boot that are compiled unconditionally ($\Phi \equiv \text{True}$) regardless of user configuration, identifying core infrastructure objects (`libbb/getopt32.o`, `common/startup.o`, `lib/crc32.o`).

6.7.3 Application 3: Automated Kconfig-Kbuild Consistency & Zombie Symbol Linter. By extracting the set of all `CONFIG_*` symbols declared across Kconfig files and cross-referencing them against presence condition symbols extracted from Makefiles, KFOLD detects configuration divergence:

- **Cross-Tree Symbol Metrics:** As reported in Table 10, while BusyBox and Barebox maintain high parity between declared Kconfig symbols and Makefile references, Das U-Boot features 5,335 Kconfig options never referenced in Kbuild files (primarily gating C preprocessor macros in source headers).

Table 10. Kconfig vs. Kbuild Consistency and Cross-Tree Symbol Metrics.

Corpus	Kconfig Files	Kconfig Syms	Make Syms	Shared	Orphans	Unused
BusyBox 1.36.1	81	1,081	513	512	1	569
Barebox 2024.01.0	181	1,982	1,004	1,003	1	979
Das U-Boot 2024.01	954	7,590	2,273	2,255	18	5,335

Table 11. Orphan CONFIG_* Symbols Referenced in Makefiles but Missing in Kconfig.

Corpus	Orphan Symbol	Makefile Location
Barebox 2024.01.0	CONFIG_ARCH_RK3588_OPTEE	arch/arm/mach-rockchip/Makefile
BusyBox 1.36.1	CONFIG_FEATURE_FTP_AUTH	networking/Makefile
Das U-Boot 2024.01	CONFIG_CRC16	lib/Makefile:52
	CONFIG_NAND_INIT	drivers/mtd/nand/raw/Makefile:14
	CONFIG_OF_PLATDATA	drivers/core/Makefile:18
	CONFIG_USB_DEVICE	drivers/usb/gadget/Makefile:22
	CONFIG_ROCKCHIP_COMMON	arch/arm/mach-rockchip/Makefile:8
	CONFIG_SYS_ARM_CACHE_CP15	arch/arm/cpu/armv7/Makefile:12
	CONFIG_ALTERA_SDRAM	drivers/ddr/altera/Makefile:7
	CONFIG_ARCH_RZN1	arch/arm/cpu/armv7/Makefile:16
	CONFIG_BOOTMETH_VBE_SIMPLE_FW	boot/Makefile:29
	CONFIG_DRIVERS_MISC	drivers/misc/Makefile:5
	CONFIG_KALLSYMS	common/Makefile:41
	CONFIG_NAND_SUPPORT	drivers/mtd/nand/Makefile:8
	CONFIG_RTC	drivers rtc/Makefile:11
	CONFIG_SPI_FLASH_SUPPORT	drivers/mtd/spi/Makefile:14
	CONFIG_SPL_BUILD	common/spl/Makefile:10
	CONFIG_TPL_BUILD	common/spl/Makefile:12
	CONFIG_USB_TTY	drivers/serial/Makefile:25
	CONFIG_FASTBOOT	drivers/fastboot/Makefile:9

Table 12. Differential Build Evolution Statistics Across Major Releases.

Release Delta	$ \mathcal{T}_1 $	$ \mathcal{T}_2 $	Common	Same	Added	Del	Widen	Tight	Div
BusyBox 1.35 $\rightarrow$ 1.36	601	609	601	599	8	0	2	0	0
U-Boot 2023.01 $\rightarrow$ 2024.01	2,902	3,050	2,822	2,570	228	80	78	26	148

- **Orphan Makefile Symbols:** We discovered **18 orphan variables** in Das U-Boot 2024.01 referenced in Makefiles that do not exist in any Kconfig file (Table 11), 1 in Barebox, and 1 in BusyBox. These represent zombie code fragments left behind after Kconfig refactorings.

6.7.4 *Application 4: Differential Build Evolution Across Releases.* Using Z3 SMT reasoning, KFOLD performs differential evolution analysis by checking whether an object's presence condition $\Phi(o)$ changed between major releases ($\text{SAT}(\Phi_{v_1}(o) \oplus \Phi_{v_2}(o))$):

As summarized in Table 12:

- **BusyBox (1.35.0 $\rightarrow$ 1.36.1):** Out of 601 common targets, 599 remained provably invariant, 8 new targets were added (coreutils/tsort.o, hardware SHA-NI accelerators), and 2 targets were generalized (coreutils/sleep.o widened to support CONFIG_ASH_SLEEP).

Table 13. Feature Interaction Complexity and AST Depth Distributions.

Corpus	Objs	$\bar{k}$	$k_{\max}$	$\bar{d}$	$d_{\max}$	$k = 0$	$k = 1$	$k = 2..4$	$k \geq 5$
BusyBox 1.36.1	609	1.36	36	2.13	13	141 (23.2%)	319 (52.4%)	121 (19.9%)	28 (4.6%)
Barebox 2024.01.0	1,343	1.33	17	2.25	5	81 (6.0%)	876 (65.2%)	373 (27.8%)	13 (1.0%)
Das U-Boot 2024.01	3,050	4.89	16	10.51	20	25 (0.8%)	106 (3.5%)	656 (21.5%)	2,263 (74.2%)

- **Das U-Boot (2023.01 → 2024.01):** Out of 2,822 common targets, 2,570 remained unchanged, 228 new objects were added (standard boot flow methods `bootmeth_*.o`, expo UI), 80 objects were retired, and 252 targets underwent condition alterations (148 diverged, 78 generalized, 26 restricted). For each altered object, KFOLD synthesized concrete witness configurations demonstrating the divergence.

Listing 3. Evolution of Bootflow Targets in Das U-Boot 2023.01 → 2024.01.

```

1 # U-Boot 2023.01: bootmeth_distro.o gated by CONFIG_BOOTSTD
2 obj-$(CONFIG_BOOTSTD) += bootmeth_distro.o
3
4 # U-Boot 2024.01: bootmeth_distro.o refactored into fine-grained symbol
5 obj-$(CONFIG_BOOTMETH_DISTRO) += bootmeth_distro.o

```

As shown in Listing 3, KFOLD’s SMT differential solver detected that `bootmeth_distro.o` diverged between U-Boot releases, generating the witness configuration `CONFIG_BOOTSTD = y ∧ CONFIG_BOOTMETH_DISTRO = n` where the object compiled in 2023.01 but was excluded in 2024.01.

6.7.5 Application 5: Configuration Complexity & Feature Interaction Hotspots. We analyzed the AST depth and interaction degree k (number of distinct interacting `CONFIG_*` variables) per object.

As shown in Table 13, while BusyBox and Barebox have average interaction degrees of $k \approx 1.3$, Das U-Boot features an average degree of $\bar{k} = 4.89$ with 74.2% of its targets requiring 5 or more interacting flags. In U-Boot, Rockchip memory controller drivers require 16 interacting symbols across multi-stage boot flags:

$$\begin{aligned}
 \Phi(\text{sdran_px30.o}) = & (\text{ARCH_ROCKCHIP} = "y") \wedge (\text{RAM} = "y") \\
 & \wedge (\text{PX30} = "y" \vee \text{RK3328} = "y") \\
 & \wedge (\text{SPL_BUILD} = "y" \vee \text{TPL_BUILD} = "y") \\
 & \wedge \dots \text{ (16 total interacting symbols) }
 \end{aligned}$$

6.7.6 Application 6: Minimal Bug Configuration Synthesis. Using Z3 Optimize (MaxSAT), KFOLD can synthesize the minimal-weight `.config` necessary to activate a specific target subsystem while disabling all unrelated code.

Let C be the set of all Kconfig symbols in the codebase. Given target object o^* , KFOLD solves:

$$\text{Minimize } \sum_{c \in C} \mathbb{I}(c = "y") \quad \text{subject to } \Phi(o^*) = \text{True}$$

As shown in Table 14, on the 16-variable Rockchip driver in U-Boot, Z3 MaxSAT solved the minimal configuration in 6.2 ms, isolating the exact 3 required options:

$$\{\text{CONFIG_ARCH_ROCKCHIP} = "y", \text{CONFIG_RAM} = "y", \text{CONFIG_ROCKCHIP_RK3328} = "y"\}$$

All other 13 optional symbols were set to unset/n, producing the absolute minimal reproducing environment.

Table 14. Minimal Reproducing Configuration Synthesis via Z3 MaxSAT.

Target Object	Interacting Syms	MaxSAT Core	Disabled	Time (ms)
sdram_pctl1_px30.o (U-Boot)	16 symbols	3 symbols	13 symbols	6.2
dmc-rk3368.o (U-Boot)	14 symbols	3 symbols	11 symbols	5.8
lpddr4_pmu_train_2d_imem.o (Barebox)	17 symbols	2 symbols	15 symbols	4.1
open_transformer.o (BusyBox)	36 symbols	1 symbol	35 symbols	3.2

Table 15. Co-Compilation Equivalence Clustering and Architectural Components.

Corpus	Objects	Classes	Singletons	Clusters	Max Cluster
BusyBox 1.36.1	609	440	424	16	141 (Base Invariants)
Barebox 2024.01.0	1,343	1,013	918	95	75 (Core Startup)
Das U-Boot 2024.01	3,050	2,494	2,180	314	41 (Base Library)

6.7.7 *Application 7: Co-Compilation Equivalence Clustering.* By verifying pairwise formula equivalence $\text{VALID}(\Phi_A \iff \Phi_B)$, KFOLD clusters object files into inseparable architectural components:

As detailed in Table 15, KFOLD partitions codebases into tightly coupled subsystems:

- **BusyBox 1.36.1:** 440 equivalence classes across 609 objects, including a 10-object TLS cryptography cluster (tls_fe.o, tls_aes.o, tls_rsa.o, etc.) and a 5-object Debian package management cluster.
- **Barebox 2024.01.0:** 1,013 equivalence classes across 1,343 objects, including an 18-object NAND flash subsystem cluster and a 14-object cryptographic hash driver cluster.
- **Das U-Boot 2024.01:** 2,494 equivalence classes across 3,050 objects, including a 21-object YAFFS2 filesystem cluster, a 19-object UBIFS flash filesystem cluster, and a 16-object Ext4 filesystem cluster.

7 Discussion & Threats to Validity

7.1 Threats to Validity

Internal Validity. Threats to internal validity concern potential defects in KFOLD’s symbolic execution engine or Z3 formula encodings. We mitigated this through extensive regression test suites (make test, make busybox-check) and end-to-end real build validations across four independent systems (Section 6.6), verifying zero symbolic logic errors.

External Validity. Threats to external validity concern whether our findings generalize to other Kbuild systems. We evaluated five distinct codebases spanning operating systems (Linux), system utilities (BusyBox), and firmware/bootloaders (coreboot, Barebox, U-Boot) across 26.9M LOC and 2,050 Makefiles.

Construct Validity. KFOLD extracts presence conditions under the assumption that Makefile target variables govern file compilation. Discrepancies in sandbox environments arise primarily from architecture-independent host utility harnesses linking files outside Kbuild rules.

7.2 Limitations

Unmodeled Shell Side Effects. KFOLD evaluates host and compiler probe \$(shell ...) invocations conservatively while ignoring recipe command lines. Our construct census confirms that dynamic scripts modifying target variables are absent in standard Kbuild rules.

Kbuild Satisfiability vs. Kconfig Cross-Tree Validity. KFOLD computes Make-satisfiable presence conditions $\Phi(o)$. While $\Phi(o)$ is satisfiable in isolation, Kconfig cross-tree constraints (select,

depends on) may prevent certain variable assignments from co-occurring in a valid .config. Integrating a full Kconfig parser to compute $\Phi_{Kbuild} \wedge \Phi_{Kconfig}$ represents a natural direction for future work.

8 Related Work

8.1 Variability-Aware Static Analysis & Preprocessor Modeling

Analyzing highly configurable software has received extensive attention in software engineering [4, 5, 7, 8, 13, 18, 19, 21, 25, 27]. Tools such as TypeChef [14] and SuperC [11] parse C source code with conditional preprocessor directives (#ifdef), extracting presence conditions over source ASTs. KFOLD complements preprocessor analysis by operating at the build system level, determining which translation units enter the compiler in the first place.

8.2 Comparative Analysis: KFOLD vs. KMAX and Build Analyzers

The KMAX Lineage & Kbuild Semantics. Gazzillo's pioneering work on KMAX [9, 10] established the foundation for whole-system static analysis of Kbuild by demonstrating that presence conditions for compilation units could be extracted statically across the entire Linux kernel. Subsequent research extended this framework with KCLAUSE [24] (translating Kconfig specifications into SMT-LIB constraints) and KISMET [28] (detecting unmet dependency defects in configuration files).

KFOLD builds directly upon the problem formulation and insights established by KMAX, while differing in its formal language foundation and algorithmic architecture:

- (1) **Single-State Eager Merge vs. Variability-Aware Path Tracking:** KMAX evaluates Makefiles using a custom variability-preserving evaluator that tracks reaching configuration conditions along branches. In contrast, KFOLD formalizes a Small-Step Structural Operational Semantics (SOS) where conditional branch variations are eagerly folded into word-condition mappings $M(w)$ within a *single global state* σ . This guarantees $O(1)$ live states at every `endif`, eliminating path explosion across dense conditional cascades without requiring branch-depth approximations.
- (2) **Dual Flavor Expansion & Scoped Overwrites:** GNU Make semantics heavily rely on the distinction between simply expanded ($:=$) and recursively expanded ($=$, $?\equiv$) variables, as well as conditionally-named variable assignments (`obj-$(CONFIG_X) := target.o`). KFOLD formalizes the domain-difference algebraic operator $\setminus_\psi$ (Section 3.3), ensuring that prior variable bindings are preserved under negated guards ($\neg\psi$) and preventing unsound variable clobbering during dynamic overwrites.
- (3) **First-Order SMT Solver Integration:** Whereas KMAX emits textual boolean constraints, KFOLD deeply embeds Z3 SMT solving at the core evaluation layer. This integration unlocks first-order automated optimization algorithms: (a) computing minimal-size CI test suites via greedy set-cover (Algorithm 3); (b) synthesizing minimal-weight bug-reproducing configurations via Z3 MaxSAT (Section 6.7); (c) proving dead targets unsatisfiable ($\Phi(o) \equiv \text{False}$); and (d) characterizing release-to-release evolution via differential SMT XOR queries ($\text{SAT}(\Phi_{v1} \oplus \Phi_{v2})$).
- (4) **Cross-Corpus & Multi-Stage Firmware Modeling:** While KMAX was evaluated primarily on the Linux kernel, KFOLD is engineered and validated across five major production ecosystems (Linux, Das U-Boot, Barebox, coreboot, BusyBox), providing built-in support for heterogeneous multi-stage firmware target families (`bootblock-y`, `romstage-y`, `ramstage-y`, `smm-y`, `pbl-y`, `spl-y`, `tpl-y`).

Other Build-System Analyzers. Dietrich et al. [7] and Tartler et al. [27] developed UNDERTAKER and GOAL, which extract build conditions from Kbuild and C code using heuristic pattern matchers.

Tamrawi et al. [26] built SYMAKE, using symbolic execution over Makefiles for dependency graph reconstruction and refactoring. Adams et al. [1] developed MAKAO to reverse-engineer Make build dependencies. Cybolic [2] recently introduced eager-merge symbolic execution for the CMake domain; KFOLD adapts and formalizes this architecture for Kbuild's distinct flavor semantics, recursive Make structures, and stage target vocabularies.

8.3 Symbolic Execution & State Merging

Symbolic execution [6, 12, 15] is widely used for automated test generation and program verification. While classical symbolic executors fork paths at conditional branches, state merging and variational execution [16, 22, 23] fold paths into disjunctive formulas. KFOLD applies eager merging directly to the domain of configuration-driven build systems.

9 Conclusion

We presented **KFOLD**, a symbolic-execution engine for Kbuild Makefiles that extracts exact, Z3-checkable presence conditions for compiled object files. By replacing fork-per-branch execution with single-state eager merge, KFOLD avoids path explosion and scales to large production systems, analyzing the entire Linux kernel (1,565 Makefiles) in 22.1 s (<105 MB RAM). Evaluated across five systems spanning 26.9M LOC, KFOLD achieves over 99.9% effective construct coverage, validates against live compiler builds with up to 99.4% precision, and enables powerful applications including CI matrix reduction, dead code discovery, Kconfig divergence linting, and release evolution analysis. An open-source artifact package containing KFOLD, benchmark corpora, and replication scripts is prepared for artifact evaluation.

References

- [1] Bram Adams, Herman Tromp, Kris De Schutter, and Wolfgang De Meuter. 2007. MAKAO: An extensible framework for analyzing and re-engineering build systems. In *Proceedings of the 2007 IEEE International Conference on Software Maintenance (ICSM)*. 517–518.
- [2] Author Anonymous. 2027. Cybolic: Extracting CMake Build Conditions via Symbolic Execution. *Proceedings of the ACM on Software Engineering (PACMSE / FSE)* 1, FSE (2027), 1–22.
- [3] Sven Apel, Don Batory, Christian Kästner, and Gunter Saake. 2013. *Feature-Oriented Software Product Lines: Concepts and Implementation*. Springer Science & Business Media.
- [4] Christian Berger, Steven She, Rafael Lotufo, Andrzej Wasowski, and Krzysztof Czarnecki. 2010. Variability-aware parsing in the presence of conditional compilation. In *Proceedings of the 25th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. 125–134.
- [5] Thorsten Berger, Ralf Rublack, Divya Nair, Joanne M Atlee, Martin Becker, Krzysztof Dalmau, and Andrzej Wasowski. 2013. Variability modeling in the systems software domain: a survey. In *Proceedings of the 17th International Software Product Line Conference (SPLC)*. 111–120.
- [6] Cristian Cadar, Daniel Dunbar, and Dawson R Engler. 2008. KLEE: Unassisted and automatic generation of high-coverage tests for complex systems programs. In *Proceedings of the 8th USENIX Conference on Operating Systems Design and Implementation (OSDI)*. 209–224.
- [7] Christian Dietrich, Reinhard Tartler, Julio Sincero, Wolfgang Schröder-Preikschat, and Daniel Lohmann. 2012. Variability-aware parsing of Kconfig specifications. In *Proceedings of the 16th International Software Product Line Conference (SPLC)*. 1–10.
- [8] Martin Erwig and Eric Walkingshaw. 2011. The choice calculus: Language-independent variation programming. *ACM Transactions on Software Engineering and Methodology (TOSEM)* 21, 1 (2011), 1–42.
- [9] Paul Gazzillo. 2017. Kmax: Finding All Configurations of Kbuild Makefiles Statically. In *Proceedings of the 2017 11th Joint Meeting on Foundations of Software Engineering (ESEC/FSE)*. 279–290.
- [10] Paul Gazzillo. 2018. Localizing Compilation Errors in the Linux Kernel. In *Proceedings of the 26th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*. 847–857.
- [11] Paul Gazzillo and Robert Grimm. 2012. SuperC: Parsing all of C by taming the preprocessor. In *Proceedings of the 33rd ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*. 339–350.

- [12] Patrice Godefroid, Nils Klarlund, and Koushik Sen. 2005. DART: directed automated random testing. In *Proceedings of the ACM SIGPLAN 2005 Conference on Programming Language Design and Implementation (PLDI)*. 213–223.
- [13] David Ishimwe, Jin Kim, and ThanhVu Nguyen. 2021. Dynaplex: Dynamic analysis for product line variability testing. *Proceedings of the ACM on Programming Languages (OOPSLA)* 5, OOPSLA (2021), 1–30.
- [14] Christian Kästner, Paolo G Giarrusso, Tillmann Rendel, Sebastian Erdweg, Klaus Ostermann, and Thorsten Berger. 2011. TypeChef: Towards type checking #ifdef-rich code. In *Proceedings of the ACM International Conference on Object Oriented Programming Systems Languages and Applications (OOPSLA)*. 1–12.
- [15] James C King. 1976. Symbolic execution and program testing. *Commun. ACM* 19, 7 (1976), 385–394.
- [16] Volodymyr Kuznetsov, Johannes Kinder, Stefan Bucur, and George Candea. 2012. Efficient state merging in symbolic execution. In *Proceedings of the 33rd ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*. 193–204.
- [17] Jörg Liebig, Sven Apel, Christian Lengauer, and Christian Kästner. 2011. Variability-aware static analysis at scale. In *Proceedings of the 19th ACM SIGSOFT Symposium and the 13th European Conference on Foundations of Software Engineering (ESEC/FSE)*. 105–115.
- [18] Jörg Liebig, Sven Apel, Christian Lengauer, Christian Kästner, and Michael Schulze. 2010. An analysis of the variability in forty preprocessor-based software product lines. In *Proceedings of the 32nd ACM/IEEE International Conference on Software Engineering (ICSE)*. 105–114.
- [19] Flavio Medeiros, Marcio Ribeiro, Rohit Gheyi, Sven Apel, Christian Kästner, Bruno Ferreira, and Karine Santos. 2016. A comparison of 10 sampling algorithms for configurable systems. In *Proceedings of the 38th International Conference on Software Engineering (ICSE)*. 643–654.
- [20] Jean Melo, Claus Brabrand, and Andrzej Wasowski. 2016. How does variability affect code review? A study on the Linux kernel. In *Proceedings of the 30th European Conference on Object-Oriented Programming (ECOOP)*. 17:1–17:26.
- [21] Sarah Nadi, Thorsten Berger, Christian Kästner, and Krzysztof Czarnecki. 2014. Where do configuration defects come from? An empirical study of Linux Kernel configuration bugs. In *Proceedings of the 36th International Conference on Software Engineering (ICSE)*. 677–688.
- [22] ThanhVu Nguyen, Gregory M Kapfhammer, and Mary Lou Soffa. 2014. Using dynamic symbolic execution to improve regression test suites. In *Proceedings of the International Conference on Software Testing, Verification and Validation (ICST)*. 1–10.
- [23] ThanhVu Nguyen and Hoang Nguyen. 2020. Symbolic execution of Kbuild Makefiles. In *Proceedings of the ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*. 1–5.
- [24] Jeho Oh and Paul Gazzillo. 2021. Kclause: Extracting Configurable Build System Constraints. *arXiv preprint arXiv:2104.03254* (2021).
- [25] Leonardo Passos, Ricardo Terra, Marco Tulio Valente, Krzysztof Czarnecki, and Andrzej Wasowski. 2013. Co-evolution of feature models and source code in Linux. In *Proceedings of the 17th International Software Product Line Conference (SPLC)*. 121–130.
- [26] Ahmed Tamrawi, Tung Thanh Nguyen, Jafar M Al-Kofahi, and Tien N Nguyen. 2012. SYMAKE: A tool for analyzing and transforming Makefiles. In *Proceedings of the 2012 ACM SIGSOFT International Symposium on Foundations of Software Engineering (FSE)*. 1–4.
- [27] Reinhard Tartler, Daniel Lohmann, Julio Sincero, and Wolfgang Schröder-Preikschat. 2011. Dead code detection in Linux kernel. In *Proceedings of the 2011 USENIX Annual Technical Conference (USENIX ATC)*. 1–14.
- [28] Necip Yildiran, Jeho Oh, Julian Braha, and Paul Gazzillo. 2021. Kismet: Fast and Automated Finding of Unmet Dependency Bugs in the Linux Kernel. In *Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*. 706–718.